PROVE-IT BUILD PACK — AGENT INSTRUCTION DOCUMENT

Open Agent Hackathon 2026 | GenAI.Works | Strictly follows attached Prove-It Build Plan v3

1. HACKATHON & PROJECT IDENTITY

Hackathon: Open Agent Hackathon 2026 (https://hackathon.genai.works/event/open-agent-hackathon-2026)

Primary Track: Track 02: The Agent That Can Explain Why OR Track 03: Reasoning Architecture (both strong; prefer Track 02 if evidence quality is the standout)

Project Name: CareSync — Family Health Evidence & Coordination Agent

One-sentence pitch: A family-facing agent that unifies fragmented health data (lab PDFs, doctor portals, pharmacy, wearables, caregiver notes) via Zetaris, maintains a longitudinal health world-model and memory via Meterless, and produces evidence-backed explanations and visit-prep packages that a non-clinical caregiver can trust.

Target end-user: Adult children and spouses caring for aging parents or family members with chronic conditions who currently juggle portals, paper, and group chats.

Hard constraints from rules: Must integrate Zetaris + Meterless. Must use Cursor for development. Integrate NVIDIA (Nemotron) where it contributes. Demo video 1-4 min. Slide deck. Public GitHub + README. Explicit sponsor integration explanation.

2. STAGE 0 — ONE-PAGE BRIEF (MANDATORY BEFORE ANY CODE)

Answer every field completely. If any answer is unclear, stop and refine the idea.

#	Field	Answer
1	Problem	Family caregivers struggle to keep a coherent picture of a loved one's health because data is fragmented across hospital portals, lab PDFs in email, pharmacy apps, wearable exports, insurance statements, and ad-hoc notes. This leads to missed patterns, unprepared doctor visits, and high cognitive load.
2	Thesis	If an agent can query fragmented health-related sources in-situ via Zetaris and maintain a longitudinal patient + family world-model via Meterless, it can surface evidence-backed patterns and prepare high-quality visit materials that reduce caregiver cognitive load and improve the signal doctors receive.
3	Invariant	The agent must never diagnose or give medical advice that replaces a clinician; it must never share data outside the family circle without explicit consent; every medical-sounding statement must carry source citations; simulated data must be labeled.
4	Why AI / agent	AI decides which documents and sources are relevant, extracts structured facts, maintains the longitudinal narrative, and drafts questions/summaries. Deterministic extraction + validation code prevents invented lab values. Human (caregiver) always reviews and decides what is shared with clinicians.
5	Load-bearing tech	Zetaris for multi-source document + structured data access; Meterless World Model + H-MEM for the longitudinal patient state and family context. Without them the system is either a single-PDF chatbot or a stateless summarizer.
6	Most dangerous assumption	That caregivers can successfully upload or connect the necessary documents/sources in a short session, and that extraction of key clinical facts (dates, values, meds) is reliable enough to be useful rather than dangerous.
7	Proof experiment	Baseline: caregiver prepares for a simulated specialist visit using only their existing method (time + completeness of key facts measured). Intervention: CareSync produces a visit-prep package. Result: higher completeness of critical facts + lower preparation time + every fact cited.
8	Demo moment	A caregiver asks 'What has changed in the last 90 days that the cardiologist should know?' and receives a cited timeline + suggested questions in under two minutes.
9	Distribution hook	A short public video showing the agent correctly linking a lab trend to a medication change using real (or clearly labeled) sample documents and producing a clean visit-prep PDF.

3. STAGE 1 — KILL-TEST (FIRST ~10% OF TIME)

Spend the first ~10% of the build window trying to kill the idea. Use throwaway scripts, not the product.

- Feed 3–5 real-looking sample lab PDFs / discharge summaries into a throwaway extraction pipeline and measure fact extraction accuracy against a hand-labeled gold set.
- Confirm Zetaris can treat uploaded PDFs / extracted tables + a simple Postgres meds list as queryable sources.
- Run Meterless World Model + H-MEM references and confirm they can store and retrieve a multi-month timeline of events.
- Test whether a non-technical caregiver can complete an 'add document' flow in under 4 minutes.
- Read all relevant Zetaris and Meterless limits, auth, and data-handling docs.
- If extraction reliability is too low or the connection UX is too hard, narrow the vertical slice to a smaller, more reliable set of document types.

If any critical assumption fails → pivot the concept immediately. Do not fight the protocol.

4. STAGE 2 — ARCHITECTURE AND VERTICAL SLICE

Build ONE complete path that demonstrates the thesis. Keep architecture small. Justify every service.

Canonical flow (label every component REAL / TESTNET / SIMULATED / DRY-RUN / READ-ONLY / MEASURED)

```
Caregiver → Auth → Upload document or ask question → Agent (Nemotron) → Meterless World Model / H-M
EM (longitudinal patient state) → Zetaris tools (document store + structured sources) → Deterministic
 fact validation → Evidence-backed answer or visit-prep package → Caregiver review → optional share
```

High-level architecture

- Frontend: Next.js + Tailwind — caregiver-friendly, large touch targets, clear 'this is not medical advice' banners.
- Document intake: simple upload → text/PDF extraction (use a reliable open-source or sponsor-friendly extractor) → structured fact candidates.
- Agent: orchestrates retrieval from Meterless, queries Zetaris, validates facts, generates explanations and visit-prep.
- Context: Meterless World Model holds the longitudinal patient state (conditions, meds, key events); H-MEM stores conversation and document memory.
- Data: Zetaris over a document metadata + extracted-facts store plus any structured sources (meds list, wearable CSV).
- Safety layer: hard-coded refusal for diagnosis / treatment advice; every clinical fact must have a source citation or it is dropped.
- Audit: full chain logged.

Vertical slice definition (must work end-to-end)

- Caregiver can create a patient profile and upload at least two document types (e.g., lab PDF + medication list).
- Agent extracts and stores key facts into the Meterless world model.
- Caregiver asks a longitudinal question or requests a visit-prep package for a named specialty.
- System returns a cited timeline / summary + suggested questions, with every clinical fact linked to its source document.
- Clear non-diagnostic disclaimer is always visible.

Deploy the slice early. Test from a clean browser, fresh account, and fresh environment.

5. STAGE 3 — BREAK IT

Attack your own product before anyone else does. Produce a 'Ways I tried to break it' test matrix.

Area	Attack vectors to implement
Input	Corrupt PDFs, scanned images with bad OCR, adversarial prompts asking for diagnosis, oversized files
Auth	Wrong family member access, session issues
State	World-model inconsistency after new document, partial writes
Network	Extraction service timeout, Zetaris or model outage
Data	Conflicting lab values from different sources, missing units, date ambiguity
AI	Invented lab numbers, dropped citations, unsafe medical advice generation

Area	Attack vectors to implement
Permissions	Any attempt to contact external clinicians or share data without explicit consent must be blocked
UX	Overwhelming medical jargon, mobile breakage, unclear next actions

The system must know when NOT to act, with a clear human escalation path.

6. STAGE 4 — PROVE IT

Every important claim needs evidence another engineer can verify.

- Removal test: without Zetaris multi-source capability the agent cannot correlate across documents; without Meterless the longitudinal view collapses.
- Measure completeness of visit-prep package against a gold checklist on 3 sample patients.
- Audit trail for the main demo path.
- Safety review: confirm zero diagnostic statements reach the user without heavy hedging and citation.

Proof pack checklist (include ALL)

- One clean end-to-end demo path
- Architecture diagram or data-flow note
- Core test results
- Live URL (or clearly labeled local + video)
- Failure-case results
- Real evidence (logs, API calls, transaction IDs, timestamps)
- Real vs simulated labels on every claim
- Known limitations and hardest engineering decision

7. REQUIRED TECH STACK (EXACT)

Use exactly these components. Do not invent alternatives unless a kill-test forces a pivot.

Layer	Technology	Why / How
Frontend	Next.js 14+ + Tailwind + shadcn/ui	Accessible caregiver UI
Agent orchestration	TypeScript or Python (LangGraph)	Stateful multi-step reasoning
Reasoning model	NVIDIA Nemotron	Long-context reasoning over health timeline; NVIDIA requirement
Data harness	Zetaris	Query across document-derived tables + structured sources
Context / memory	Meterless (World Model + H-MEM)	Longitudinal patient state and family context
Document processing	pdfplumber / PyMuPDF + simple LLM extraction or rule-based for labs	Turn PDFs into structured candidates
Database	Postgres	App state, document metadata, audit
Auth	NextAuth / Clerk with simple family invite	Caregiver-friendly
Deployment	Vercel + backend host or Docker Compose	Clear REAL/SIMULATED labeling
Development	Cursor (mandatory)	Primary IDE

8. ENVIRONMENT VARIABLES YOU MUST PROVIDE

Create a .env file. Never commit secrets. Document every variable in README.

Variable	Description	Required?
ZETARIS_JDBC_URL or ZETARIS_MCP_ENDPOINT	Zetaris connection	Yes
ZETARIS_API_KEY	Zetaris auth	Yes

Variable	Description	Required?
NVIDIA_API_KEY or NEMOTRON_ENDPOINT	NVIDIA Nemotron access	Yes
DATABASE_URL	Postgres	Yes
NEXTAUTH_SECRET / CLERK_SECRET_KEY	Auth	Yes
METERLESS_DATA_DIR	Local Meterless storage path	Yes
DEMO_MODE	Force SIMULATED labels	Recommended
MAX_UPLOAD_MB	Upload size limit	Recommended

9. PRECISE BUILD STEPS (EXECUTE IN ORDER)

Follow these steps sequentially. Do not skip. Mark each completed step.

Step 1: Confirm Zetaris and Meterless access. Prepare 4–6 realistic (de-identified or synthetic) sample health documents.

Step 2: Initialize Cursor project with Next.js + auth + Postgres.

Step 3: Implement document upload + basic text/PDF extraction pipeline. Store extracted candidates.

Step 4: Stand up Zetaris over the extracted-facts store + a simple meds table.

Step 5: Integrate Meterless World Model and H-MEM; define the patient state schema.

Step 6: Build the agent loop with hard safety refusals for diagnosis.

Step 7: Implement visit-prep generation and the 'what changed' query.

Step 8: Add full audit logging and REAL/SIMULATED labels.

Step 9: Run break matrix, especially safety and extraction failure cases.

Step 10: Execute proof experiment on sample patients.

Step 11: Package README, video, slides, sponsor explanation.

Step 12: Final gate check and submit.

10. TEST PLAN — EVERY TEST MUST PASS BEFORE SUBMISSION

Write automated tests where possible. Record manual test results with screenshots/logs.

ID	Test	Pass criteria
T01	Lab PDF extraction recovers date, test name, value, unit for at least 80% of key fields on sample set	Measured against gold labels
T02	Meterless World Model updates and retrieves a 90-day event timeline	Events appear in correct order
T03	Agent refuses pure diagnostic questions with a clear, helpful redirect	No diagnosis emitted
T04	Every clinical fact in a visit-prep package has a source citation	No uncited facts
T05	Conflicting values from two sources are surfaced as conflicts, not silently averaged	Conflict visible to user
T06	DEMO_MODE forces SIMULATED labels	Labels present
T07	Audit trail complete for main path	IDs + timestamps
T08	Non-technical caregiver can upload a document and get a visit-prep in under 5 minutes	Timed success
T09	Removal of Meterless collapses longitudinal answers	Graceful degradation message
T10	No PHI leaves the system in logs or prompts beyond what is required	Review clean

11. PRODUCTION-READY CHECKLIST

• Prominent 'This is not medical advice / not a diagnosis' banner on every clinical screen
• All clinical facts cited or dropped
• REAL vs SIMULATED labels everywhere
• No external sharing without explicit caregiver confirmation
• README documents privacy posture and limitations clearly
• Demo video shows citation UI and safety behavior
• Slide deck highlights unique Zetaris (multi-document + structured) and Meterless (longitudinal world model) usage
• GitHub public and reproducible

12. SUBMISSION REQUIREMENTS (MUST SATISFY ALL)

• Demo video 1–4 minutes showing the main workflow, how the agent solves the selected challenge, key features, and explicit Zetaris + Meterless integration.
• Slide deck covering: problem + target users, solution, agent workflow/architecture, technical stack, how Zetaris and Meterless were used, what was unique about sponsor tech usage, product visuals, future enhancements.
• Public GitHub repository containing all source code and a complete README (setup, usage, dependencies, environment variables, known limitations).
• Clear written explanation (in README or separate doc) of: how Zetaris was integrated, how Meterless was integrated, how Cursor was used during development, where NVIDIA technology contributes, and what was unique about the use of sponsor technologies.
• All claims labeled REAL / SIMULATED / DRY-RUN / MEASURED.
• Audit trail present for the main happy path (input → evidence → decision → tool call → validation → action → result with IDs and timestamps).

13. WHAT YOU PERSONALLY MUST DO BEFORE SUBMISSION

• Prepare a small set of realistic, de-identified or fully synthetic health documents you are willing to show in a public demo.
• Confirm Zetaris can handle the document-derived tables you plan to create.
• Decide the primary track early.
• Personally verify the safety refusals — do not rely solely on the model.
• Record a clear demo video that shows both the evidence trail and the non-diagnostic behavior.
• Write the sponsor integration explanation yourself.
• Complete the 9 gates honestly.
• Submit before 27 Oct 2026 23:45 UTC.

14. GATES — EVERY ONE GETS A DELIBERATE DECISION

Gate	Check	Passed?
1	Problem explained in 20 seconds	[ ]
2	Thesis in one sentence	[ ]
3	Dangerous assumption tested	[ ]
4	Core flow works end-to-end	[ ]
5	Failure modes tested	[ ]
6	Claims reproducible	[ ]
7	Stranger can use it unaided	[ ]
8	Demo lands in 2–3 minutes	[ ]
9	Public post creates curiosity	[ ]

15. TIME BUDGET (RESHAPE TO YOUR REALITY)

Phase	% of time	Focus
Brief + kill-test	20%	Stage 0 + Stage 1

Phase	% of time	Focus
Slice + deploy	35%	Vertical slice + real integrations
Break	15%	Attack matrix + hardening
Prove	15%	Evidence, removal test, metrics
Package + demo	15%	README, video, slides, marketing

CRITICAL AGENT INSTRUCTION

This document is intentionally verbose and sequential. Execute steps in order. Do not invent new components, skip tests, or present simulations as production results. Label every claim. If a step is blocked, stop and report the exact blocker with evidence. The goal is a verifiable, production-oriented agent that does real work for a real end-user, not a flashy prototype.